

Shibboleth SSO 学習環境 構築手順書

第 0.13 版

2026-07-04

本書は、PLM システム（顧客環境：Windows Server + IIS）の Shibboleth 認証連携を理解するための、自宅 PC 上の小規模学習環境の構築手順をまとめたものです。検証で得た知見を、組織の検証環境構築にフィードバックすることを目的とします。

0. 改訂履歴

版	日付	内容	備考
0.1	2026-06-30	初版。全体構成とフェーズ 1（WSL2 有効化）を記載	以降フェーズは順次追記
0.2	2026-07-01	スリープ無効化を前提化し時刻同期手順を簡素化。着手前の運用チェックポイント取得を追記。メモリを静的 8GB 前提に統一	復帰時再同期は付録 C へ移動
0.3	2026-07-01	フェーズ 2（ネットワーク方式選定・hosts・内部 CA/TLS 証明書）を追記。証明書 2 種類の参考解説を追加。.wslconfig の autoMemoryReclaim を [experimental] へ修正。オフライン導入(付録 E)を追加	段階アプローチ(A→B)前提
0.4	2026-07-01	フェーズ 3（OpenLDAP：ディレクトリ設計・テストユーザー・検索バインド）を追記。PLM へ渡す識別子は個人番号(uid)のみとする方針を反映	NameID に個人番号を載せる前提
0.5	2026-07-01	フェーズ 4（OpenJDK 17 + Tomcat 10.1）を追記。識別子の表現を「個人番号」に統一（秘匿情報保護）。フェーズ 3 に LDIF 作業ディレクトリ（例 ~/lab-ldap）を追記。本文	—

版	日付	内容	備考
		フォントを約 2pt 縮小 (本文 10pt)	
0.6	2026-07-02	フェーズ 5 (Shibboleth IdP 5 導入・Tomcat 配置・LDAP 認証) を追記。\$8.2 に Tomcat 展開確認・chmod の sudo bash 化・権限注記を補足。Word 版フォントを Meiryo に設定 (PDF は環境制約により Noto 系で生成)	IdP ホーム=/opt/shibboleth-idp
0.7	2026-07-02	フォントを Noto 系に統一 (Word/PDF 一致)。\$9.1 を latest5 実在版確認 + curl -L + 検証に修正 (例示 5.2.3)。\$9.3 の chmod を sudo bash 化。\$9.4 を secrets.properties 集約・dnFormat 修正・重複回避に更新	実機検証の知見を反映
0.8	2026-07-02	フェーズ 6 (Apache HTTPD 前段・8443/TLS 終端・Tomcat 8080 ヘリバースプロキシ) を追記。ブラウザから IdP へ HTTPS 到達可能に	idp 証明書(a)を初めて使用
0.9	2026-07-03	フェーズ 7 (IIS 導入・443/TLS バインド・確認ページ) を追記。実機知見として「(A) の localhost 転送が効かない場合は mirrored へ移行」を \$6.3・\$10.5 に反映	mirrored で (B) 相当へ前倒し移行
0.10	2026-07-03	フェーズ 8 (Shibboleth SP 3 導入・IIS 連携・サイト全体保護・SP 鍵(b)生成) を追記。\$11.5 にビルトイン Administrator の承認モード差異 (Server 2016 と Win11) と ASP 文字コードの補足を追記	NameID=REMOTE_USER 標準形
0.11	2026-07-04	フェーズ 9 (メタデータ相互登録・IdP エンドポイントの:8443 補正・初回 SSO 成立) を追記。\$1	静的メタデータ方式

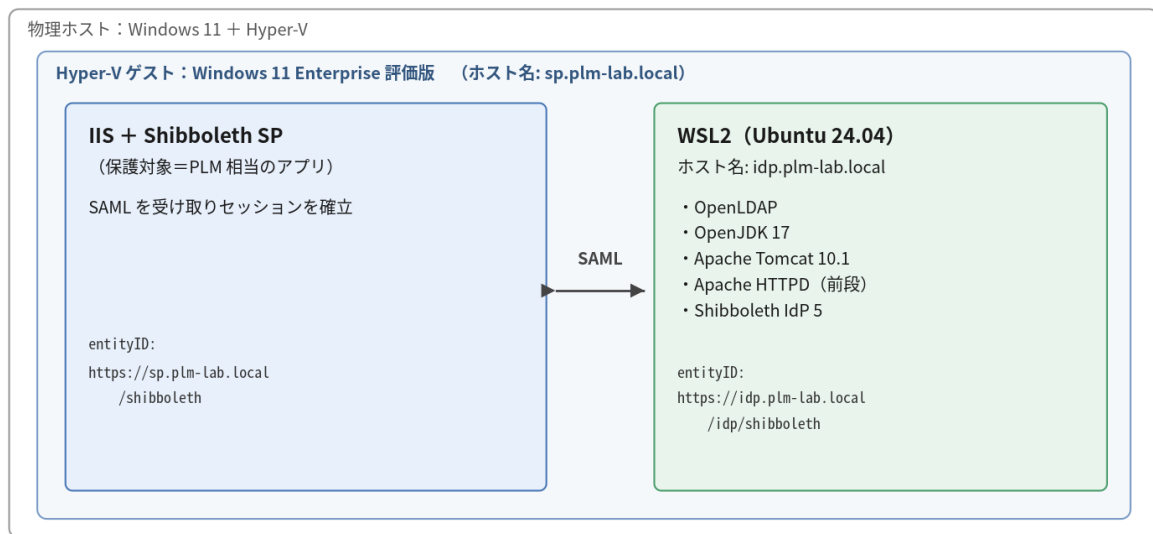
版	日付	内容	備考
		2.1に配置注記（C:）、\$1 2.2をネイティブモ ジュール(ShibNative)確 認に修正	
0.12	2026-07-04	フェーズ10（個人番号を NameIDで渡しREMOTE_US ERへ）を追記し 全SSO完 成 。\$13.2の<MetadataProv ider>配置を訂正（</Sess ions>の後）、\$10.4にmir rored時のListen 80/443 無効化を追記、\$13.4に反 映はTomcat再起動が確 実と注記	NameID方式で個人番号 を連携
0.13	2026-07-04	フェーズ11（結合テス ト・ログ・再起動堅牢 性・問題早見表・本番展 開メモ）を追記し 全工程 完了 。\$14.1を訂正（uid はPrincipalNameで定義 済み・attribute-resolver.x ml変更不要）	全11フェーズ完了

1. 目的と前提

- ・ **目的**：ShibbolethによるSAML SSO（IdP ← SP）の仕組みを、手作業構築を通じて理解する。
- ・ **方針**：Dockerを使わず、できる限り手作業で構築する。
- ・ **物理ホスト**：自宅PC（Windows 11、Hyper-V有効）。
- ・ **検証用 Windows**：Hyper-V 仮想マシン上の **Windows 11 Enterprise 評価版**（90日／本書執筆時点で rearm 残回数2）。
- ・ **役割分担**：
 - **WSL2（Ubuntu）**：OpenLDAP / OpenJDK / Apache Tomcat / Apache HTTPD / Shibboleth IdP
 - **Windows 仮想マシン本体**：IIS / Shibboleth SP
- ・ このWSL2は上記Windows仮想マシンの「中」で動作する（＝入れ子の仮想化）。
- ・ **前提（電源管理）**：物理ホスト・ゲスト仮想マシンとも **Windowsのスリープ／休止は無効化**していること（組織の対象PCも同様の運用）。本書はこの前提に立ち、スリープ復帰に伴う時刻ずれ対策は主手順から除外する（スリープを使う環境向けの参考手順は付録C）。

⚠ 重要な前提：WSL2のLinux環境はこのWindows仮想マシン内に格納されます。仮想マシンを削除するとIdP側（LDAP / Tomcat / IdP）も全て失われます。評価版の期限は **削除・再インストールではなく slmgr /rearm で延長**してください（付録A）。作業成果は付録Bの方法で保全します。

2. 全体アーキテクチャ



全体アーキテクチャ

SAML Web SSO の流れ（ブラウザ経由のリダイレクト）：

1. ブラウザが SP（IIS）の保護対象リソースにアクセスする。
2. SP がブラウザを IdP（WSL2）へリダイレクトする。
3. IdP が OpenLDAP に対してユーザーを認証する。
4. IdP が SAML アサーションを、ブラウザ経由で SP の ACS エンドポイントへ POST する。
5. SP がセッションを確立し、PLM（保護対象）が表示される。

ブラウザは **SP と IdP の両方**に、一貫したホスト名・URL・TLS 証明書で到達できる必要があります（ネットワーク設計はフェーズ 2 で扱います）。

3. パラメーター一覧（確定・調整用）

本書全体で参照する値。**着手前に内容を確認**し、変更する場合はここを起点に各フェーズへ反映してください。

項目	既定値	備考
内部ドメイン	plm-lab.local	学習用の架空ドメイン
IdP ホスト名	idp.plm-lab.local	WSL2 (Ubuntu) 側
SP ホスト名	sp.plm-lab.local	Windows (IIS) 側
Ubuntu バージョン	24.04 LTS	WSL2 ディストリ
Java	OpenJDK 17	Shibboleth IdP 5 の要件
Servlet コンテナ	Apache Tomcat 10.1.x	IdP 5 は Tomcat 10.1 必須 (9 系不可)
IdP	Shibboleth IdP 5.x	Java 17 / Jakarta 名前空間
SP	Shibboleth SP 3.x (ISAPI)	IIS 用
IdP entityID	https://idp.plm-lab.local/idp/shibboleth	フェーズ 5 で確定

項目	既定値	備考
SP entityID	https://sp.plm-lab.local/shibboleth	フェーズ 8 で確定
LDAP ベース DN	dc=plm-lab,dc=local	OpenLDAP (フェーズ 3)
LDAP 管理者 DN	cn=admin,dc=plm-lab,dc=local	slapd 管理者
LDAP 検索バインド	uid=idp-reader,ou=people,dc=plm-lab,dc=local	IdP の LDAP 検索用 (読取専用)
テストユーザー	uid=90001 / uid=90002	ou=people 配下。uid=個人番号 (PLM へ渡す識別子)
IdP ホーム	/opt/shibboleth-idp	Shibboleth IdP 5 (フェーズ 5)
SP ポート (HTTPS)	443	IIS (SP)
IdP ポート (HTTPS)	8443	WSL2 の Apache 前段 (同一 IP での 443 衝突回避)
内部 CA ルート名	PLM-Lab Root CA	フェーズ 2 で作成。idp/sp のサーバ証明書を発行
ネットワーク方式	(A)-(B) 段階移行	(A)=NAT+localhost、(B)=mirrored (フェーズ 2 で選定)
ゲスト VM メモリ	8 GB (固定)	入れ子仮想化のため動的メモリは無効化
ゲスト VM vCPU	4	目安
タイムゾーン	Asia/Tokyo	ゲスト・WSL2 とも統一

ホスト名は当面 hosts ファイルで解決します (DNS サーバは立てません)。WSL2 の IP は既定 (NAT) では動的なため、IP は固定値として表に載せず、フェーズ 2 のネットワーク設計で扱います。

4. 構築フェーズ全体像 (ロードマップ)

フェーズ	内容	本書での状態
1	WSL2 の有効化と Ubuntu 導入・時刻同期	✓ 本版で記載
2	ネットワーク方式の選定 / hosts / 内部 CA ・ TLS 証明書	✓ 本版で記載
3	OpenLDAP 構築 (ユーザーディレクトリ)	✓ 本版で記載
4	OpenJDK 17 + Tomcat 10.1 導入	✓ 本版で記載
5	Shibboleth IdP 5 インストール ・ LDAP 連	✓ 本版で記載

フェーズ	内容	本書での状態
	携	
6	Apache HTTPD (IdP 前段のリバースプロキシ)	✓ 本版で記載
7	IIS 構築 (SP の保護対象)	✓ 本版で記載
8	Shibboleth SP (ISAP I) インストール	✓ 本版で記載
9	メタデータ交換 (IdP → SP の相互信頼)	✓ 本版で記載
10	属性連携 (個人番号を NameID → SP の REMOTE_USER)	✓ 本版で記載
11	結合テスト (SSO ログイン・ログアウト・属性確認)	✓ 本版で記載

各フェーズは「目的 → 前提 → 手順 → 動作確認」の順で記載します。

5. フェーズ 1：WSL2 の有効化と Ubuntu 導入

目的：Hyper-V ゲストの Windows 11 上で WSL2 を有効化し、Ubuntu 24.04 を稼働させ、以降の IdP スタックの土台を整える。あわせて、入れ子構成で問題になりやすい**時刻同期**を最初に固めておく。

5.1 事前確認

- ・ 物理ホストで Hyper-V が有効で、ゲストの Windows 11 評価版が作成済みであること。
- ・ ゲストに割り当てるメモリ (8GB 目安) の余裕が物理ホストにあること。
- ・ 以降「物理ホスト側」と「ゲスト側」で実行場所が分かれる点に注意。

5.2 物理ホスト側：メモリ設定・入れ子仮想化の有効化・着手前チェックポイント

WSL2 自体が軽量 VM として動くため、ゲスト Windows に**ネステッド仮想化を公開**する必要があります。**ゲスト VM を「シャットダウン」してから** (一時停止・保存状態ではなく完全な電源オフ)、物理ホストの**管理者権限 PowerShell** で実行します。

1) 対象 VM 名を確認

```
Get-VM
```

2) 入れ子仮想化を有効化 (VM は停止状態であること)

```
Set-VMProcessor -VMName "<VM名>" -ExposeVirtualizationExtensions $true
```

3) 動的メモリを無効化し固定 8GB にする (入れ子仮想化との相性のため)

```
Set-VMemory -VMName "<VM名>" -DynamicMemoryEnabled $false -StartupBytes 8GB
```

4) ネットワークの MAC アドレススプーフィングを有効化 (入れ子のネットワーク用)

```
Set-VMNetworkAdapter -VMName "<VM名>" -MacAddressSpoofing On
```

5) 電源断時に保存状態にせずシャットダウンさせる (保存状態からの復帰による時刻ずれ回避)

```
Set-VM -VMName "<VM名>" -AutomaticStopAction ShutDown
```

6) 確認

```
Get-VMProcessor -VMName "<VM名>" | Select-Object ExposeVirtualizationExtensions
```

<VM名> は手順1で確認した実際の名前に置き換える。ExposeVirtualizationExtensions が True になっていれば OK。物理ホスト側で、ゲストの **Hyper-V 統合サービス「時刻同期」** が有効であることも確認 (既定で有効)。これによりゲスト Windows はホストの時計に追従します。△ 入れ子仮想化を有効にすると **メモリは実質的に固定** になります (動的メモリは変動せず、量を変えるには VM の停止が必要)。フェーズ 4 以降のフルスタック稼働を見込み、本書では **静的 8GB** を前提とします。

続いて、やり直しに備えた **着手前の復元ポイント** を取得します。入れ子のハイパーバイザーを載せる親 VM では、**稼働中のスタンダード (メモリ状態) チェックポイント** は利用できないため、**VM を停止したまま、種類を運用 (Production) にして取得** します。

種類を運用チェックポイントに固定 (既定だが明示)

```
Set-VM -VMName "<VM名>" -CheckpointType Production
```

フェーズ1着手前の素の状態を取得 (VMは停止中のまま)

```
Checkpoint-VM -VMName "<VM名>" -SnapshotName "Phase1 前_素の Windows11"
```

チェックポイントは短期のやり直し用であり、バックアップではありません (付録 B)。以降、稼働中に取得する場合は直前に `wsl --shutdown` を実行し、WSL2 の仮想ディスクを静止させてから取得してください。

5.3 ゲスト側：WSL2 の有効化

ゲストの Windows 11 を起動し、**管理者権限のターミナル (PowerShell)** で実行します。

WSL 一式 (WSL2 + 既定の Ubuntu) を導入。必要な Windows 機能も自動で有効化される

```
wsl --install
```

- 再起動を求められたら **再起動**。再起動後、Ubuntu の初期セットアップが自動で立ち上がる場合があります。
- ディストリを明示したい場合：

```
wsl --install -d Ubuntu-24.04
```

wsl --install が使えない古いビルドの場合は、機能を手動で有効化：

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart
```

```
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
```

再起動後

```
wsl --set-default-version 2
```

```
wsl --install -d Ubuntu-24.04
```

導入後の確認：

```
wsl --version
```

```
wsl -l -v # NAME=Ubuntu-24.04, STATE=Running, VERSION=2 を確認
```

⚠ VERSION が 2 であること (1 の場合は `wsl --set-version Ubuntu-24.04 2` で変換)。

WSL2 のメモリ上限を明示（WSL2 がゲストのメモリを取り込みすぎて Windows を圧迫するのを防ぐ）。ゲスト Windows の `C:\Users\<ユーザー名>\.wslconfig` を作成し、`wsl --shutdown` で反映します（静的 8GB 前提で上限 4GB）。`autoMemoryReclaim` は **[experimental] セクション**に置きます（`[wsl2]`に置くと Unknown key 警告になります）。

```
[wsl2]
memory=4GB
processors=2
swap=2GB

[experimental]
autoMemoryReclaim=gradual
```

5.4 Ubuntu の初期設定

Ubuntu を起動（スタートメニュー → Ubuntu）。初回は UNIX ユーザー名とパスワードを設定します。以降は Ubuntu シェル内で実行：

```
# パッケージ更新
sudo apt update && sudo apt -y upgrade

# 基本ツール
sudo apt -y install curl wget vim ca-certificates net-tools
```

5.5 systemd の有効化

時刻同期サービスや今後のサービス管理のため、WSL2 で systemd を有効化します。

```
sudo tee /etc/wsl.conf >/dev/null <<'EOF'
[boot]
systemd=true
EOF
```

設定を反映するため、**Windows 側（PowerShell）**で WSL を再起動：

```
wsl --shutdown
```

Ubuntu を再度起動し、確認：

```
ps -p 1 -o comm= # → systemd と表示されること
systemctl is-system-running # → running または degraded
```

5.6 時刻同期の設定

SAML は IdP と SP の時刻の一致に敏感で、ズレると「clock skew」エラーの原因になります。本書は **スリープ無効化を前提**（\$1）とするため、スリープ復帰に伴う大きな時刻ずれは発生しない想定です。したがって、次のベースライン設定で十分です。

WSL2 内で時刻同期デーモンを常駐させます。

```
sudo apt -y install systemd-timesyncd
sudo systemctl enable --now systemd-timesyncd
sudo timedatectl set-timezone Asia/Tokyo
timedatectl # "System clock synchronized: yes" / Time zone=Asia/Tokyo を確認
```


これに加え、\$5.2 で確認した **Hyper-V 統合サービス「時刻同期」** によりゲスト Windows がホスト時計に追随し、IdP (WSL2) と SP (ゲスト Windows) はいずれも同じ物理ホストの時計に由来するため、相対ズレは実用上ごく小さく保たれます。

補足：学習環境がインターネットに出られず外部 NTP に届かない場合 (synchronized=no) でも、上記の理由で相対ズレは問題になりません。万一ずれた際の手動の応急処置は Ubuntu 内で `sudo hwclock -s`。スリープを無効化していない環境で運用する場合は、復帰時の自動再同期タスク (付録 C) を追加してください。

5.7 動作確認チェックリスト

#	確認内容	コマンド / 方法	期待結果
1	WSL2 が稼働	<code>wsl -l -v</code> (Windows 側)	Ubuntu-24.04 / Running / 2
2	WSL バージョン	<code>wsl --version</code>	バージョンが表示される
3	OS バージョン	<code>lsb_release -a</code> (Ubuntu 内)	Ubuntu 24.04 LTS
4	systemd 稼働	<code>ps -p 1 -o comm=</code>	systemd
5	時刻同期	<code>timedatectl</code>	synchronized: yes (オフライン時は(b)で代替)
6	タイムゾーン	<code>timedatectl</code>	Asia/Tokyo
7	外部疎通	<code>curl -I https://archive.ubuntu.com</code> (オンライン時)	HTTP 応答が返る
8	時計の一致	Windows Get-Date と Ubuntu date を比較	数秒以内のズレ

すべて期待結果になれば、フェーズ1は完了です。この時点で付録Bのバックアップ (チェックポイント+`wsl --export`) を取得しておくことを推奨します。

6. フェーズ2：ネットワーク土台 (方式選定・hosts・内部 CA/TLS 証明書)

目的：コンポーネント導入の前に、全体が依存する「ブラウザからの到達性」「名前解決」「TLS 証明書」を先に確定する。これらは (A) と (B) で同じ値のため、ここで固めておけば段階移行時のやり直しを防げる。

6.1 ネットワーク方式の選定 ((A)~(C) とポリシー整理)

検証で使うブラウザの位置により、必要なネットワーク方式が変わります。組織展開時の判断材料として、選択肢を整理します。

選択肢	ブラウザの位置	検証できること	WSL2 方式	ポート転送の要否
(A)	ゲスト Windows 上	SAML の成立・属性連携・設定の正しさ	NAT+localhost フォワーディング	不要
(B)	物理ホスト	(A)+別マシンからの到達性 (DNS/証明書/FW)	mirrored	不要 (Hyper-V ファイアウォール許可のみ)

選択肢	ブラウザの位置	検証できること	WSL2 方式	ポート転送の要否
(C)	LAN 上の別マシン	組織ネットに最も近い到達性	mirrored + External vSwitch	不要（同上）

本書の方針：まず (A) で SAML を成立させ、その後 (B) に切り替える段階アプローチを採る。(A) は既定の NAT+localhost フォワーディングで足り、mirrored は (B) 移行時に有効化する。

重要（ポリシー）：LAN へ WSL2 を見せる方法は2通りあり、意味が異なります。NAT のまま netsh interface portproxy で見せると、これは文字どおりの**ポート転送**です。一方 **mirrored モード**では WSL 2 がルーティング可能な LAN IP を直接持ち、ポート転送なしで到達できます（設定するのは Hyper-V ファイアウォールのインバウンド許可であり、ポート転送ではありません）。ポート転送は SSO 構成に本質的に必要なものではなく、「WSL2 を NAT の内側に隠す」選択の副産物です。組織の本番（顧客 IdP と PLM/SP が別マシン＝別 IP）では、そもそもポート転送は登場しません。

6.1.1 組織管理者へ確認すべきこと（ポートフォワーディング／ネットワーク）

「ポートフォワーディング禁止」が何を対象にするかは規程により異なります。組織展開の設計前に、以下を確認してください。

1. **境界機器（ルータ／FW）でのポート開放・NAT 転送**が禁止対象か（通常はこれが主対象）。
2. **ホスト内の netsh interface portproxy**（ホスト内ポート転送）が禁止対象か。
3. **Hyper-V ファイアウォールのインバウンド許可ルール**（New-NetFirewallHyperVRule）は許容されるか。
4. **WSL2 mirrored モードの利用可否**（社内標準・セキュリティ製品・VPN／DNS との干渉の有無）。
5. **External（ブリッジ）vSwitch でゲスト VM に LAN IP を付与することの可否**（(C) 実現に必要）。

上記のうち、mirrored モードで (B)/(C) を実現する場合に使うのは 3・4・5 であり、1・2 のポート転送は用いません。この整理を提示できると、担当者との合意形成が早くなります。

6.2 ポート設計

同一ホスト（(A) の localhost、(B) の mirrored 共有 IP）で SP と IdP が 443 を取り合わないよう、IdP を別ポートにします。

役割	ホスト名	URL（ベース）	ポート	バインド先
SP	sp.plm-lab.local	https://sp.plm-lab.local/	443	IIS
IdP	idp.plm-lab.local	https://idp.plm-lab.local:8443/idp/	8443	WSL2 の Apache 前段

SAML のエンドポイントはすべて URL なので、ポートが異なっても問題ありません。組織の本番では IdP と SP が別 IP になるため、両方とも 443 を使えます（その場合はメタデータのポートを 443 に直すだけの差分）。

6.3 名前解決（hosts）

DNS サーバは立てず、hosts で解決します。(A) の段階では次を設定します。

ゲスト Windows：C:\Windows\System32\drivers\etc\hosts（管理者権限で編集）

```
127.0.0.1 sp.plm-lab.local
127.0.0.1 idp.plm-lab.local
```

WSL2 (Ubuntu) : /etc/hosts

```
echo -e "127.0.0.1\tsp.plm-lab.local\n127.0.0.1\tidp.plm-lab.local" | sudo tee -a /etc/hosts
```

(B) 移行時の変更点 (参考) : ゾンビ的な二重定義を避けるため、(B) ではゲスト Windows の hosts を「127.0.0.1」から「ゲスト VM の実 IP」に向け直します (証明書はホスト名にひも付くため作り直し不要)。この変更はフェーズ 2 の再訪で扱います。

実機知見 (重要) : 入れ子構成では、(A) の前提である **WSL2 の localhost 転送が効かないことがあります** (Windows から 127.0.0.1:<port> に繋がらない / ブラウザで「接続が拒否されました」)。切り分けは、ゲスト Windows の PowerShell で Test-NetConnection -ComputerName 127.0.0.1 -Port 8443 (False なら転送が効いていない) と、WSL2 実 IP (ip addr show eth0) に対する同コマンド (True なら Apache は正常) で行えます。この場合は **mirrored モードへ移行**するのが確実です (C:\Users\<ユーザー名>\.wslconfig の [wsl2] に networking Mode=mirrored を追加 → wsl --shutdown → 再起動)。mirrored では localhost からそのまま到達でき、実質 (B) 相当の構成になります。ブラウザから SP(443) と IdP(8443) の双方に到達する必要が出るフェーズ 7 以降を見据えると、この段階で mirrored 化しておくことが以降が楽です。

6.4 【参考】この構成で登場する 2 種類の証明書

Shibboleth では性格の異なる 2 種類の証明書が登場し、混同しやすいポイントです。役割を分けて理解してください。

観点	(a) TLS/HTTPS サーバ証明書	(b) SAML 署名・暗号化証明書
目的	ブラウザ ↔ サーバの HTTPS 通信の暗号化とサーバ真正性	SAML メッセージ (アサーション等) の署名・暗号化
使う場所	IIS (SP, 443)、Apache (IdP 前段, 8443)	IdP・SP の SAML 処理内部
誰が検証するか	ブラウザ	相手方の IdP / SP
信頼の確立方法	CA を信頼ストアに登録 (PKI)	メタデータ交換で公開鍵を相互登録
ホスト名 (SAN) 一致	必須	不要
CA 署名	必要 (本書では内部 CA で発行)	不要 (自己署名が通常)
有効期間	短め (~825 日)	長め (数年)
準備するタイミング	フェーズ 2 (本節)	各コンポーネント導入時に自動生成 (IdP=フェーズ 5、SP=フェーズ 8)

観点	(a) TLS/HTTPS サーバ証明書	(b) SAML 署名・暗号化証明書
本書での扱い	内部 CA で idp/sp 分を発行 (6.5)	自動生成のものをそのまま使用

要するに、(a) は「ブラウザの鍵マーク」のため、(b) は「SAML の相互信頼」のためのものです。本節 (6.5) で用意するのは (a) のみで、(b) は後続フェーズでコンポーネントが自動生成します。両者は独立で、(b) に CA 信頼やホスト名一致は不要です。

6.5 内部 CA の作成と idp/sp サーバ証明書の発行 (a) の準備

WSL2 (Ubuntu) 上で openssl を使い、内部 CA ルートと各サーバ証明書を作成します。

```
mkdir -p ~/lab-ca && cd ~/lab-ca
```

1) 内部 CA ルート (秘密鍵と自己署名証明書、10 年)

```
openssl genrsa -out rootCA.key 4096
openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 3650 \
  -subj "/C=JP/O=PLM-Lab/CN=PLM-Lab Root CA" -out rootCA.crt
```

2) サーバ証明書を発行する関数の手順 (idp と sp で繰り返す)

ここでは idp を例に。sp は "idp" を "sp" に置換して同様に実行。

```
cat > idp.ext <<'EOF'
subjectAltName = DNS:idp.plm-lab.local
extendedKeyUsage = serverAuth
EOF
openssl genrsa -out idp.key 2048
openssl req -new -key idp.key -subj "/C=JP/O=PLM-Lab/CN=idp.plm-lab.local" -out idp.csr
openssl x509 -req -in idp.csr -CA rootCA.crt -CAkey rootCA.key -CAcreateserial \
  -out idp.crt -days 825 -sha256 -extfile idp.ext
```

sp も同様 (SAN=DNS:sp.plm-lab.local、CN=sp.plm-lab.local)

信頼登録 ((A) の段階) :

- WSL2 (Ubuntu、curl 等の検証・Apache 用) :

```
sudo cp rootCA.crt /usr/local/share/ca-certificates/plm-lab-root-ca.crt
sudo update-ca-certificates
```

- ゲスト Windows (Edge/Chrome は Windows 証明書ストアを使用) : rootCA.crt を「信頼されたルート証明機関」に登録。WSL2 のファイルは \\wsl\$\\Ubuntu-24.04\\home\\infodba\\lab-ca\\ から取得できます。管理者 PowerShell で :

```
Import-Certificate -FilePath "\\wsl$\\Ubuntu-24.04\\home\\infodba\\lab-ca\\rootCA.crt" `
  -CertStoreLocation Cert:\\LocalMachine\\Root
```

IIS 用の補足 : IIS のバインドには証明書+秘密鍵を PFX 形式で取り込みます。フェーズ 7 で使うため、sp 証明書を PFX 化しておきます。

```
openssl pkcs12 -export -out sp.pfx -inkey sp.key -in sp.crt -certfile rootCA.crt
```

(B) 移行時 : 物理ホストの証明書ストアにも同じ rootCA.crt を登録すれば、ホスト名にひも付く証明書はそのまま使えます (作り直し不要)。

6.6 動作確認チェックリスト（フェーズ2）

#	確認内容	方法	期待結果
1	hosts 解決	ゲスト Windows で ping idp.plm-lab.local	127.0.0.1 に解決
2	CA・証明書生成	ls ~/lab-ca (Ubuntu)	rootCA.crt / idp.crt / sp.crt 等が存在
3	SAN の確認	openssl x509 -in idp.crt -noout -text \ grep -A1 "Subject Alternative"	DNS:idp.plm-lab.local
4	WSL2 の CA 信頼	openssl verify -CAfile rootCA.crt idp.crt	idp.crt: OK
5	Windows の CA 信頼	certlm.msc → 「信頼されたルート証明書機関」	PLM-Lab Root CA が存在

この時点では、証明書を実際に使う Web サーバ（Apache/IIS）はまだ無いため、HTTPS 応答の確認は各コンポーネント導入後（フェーズ6・7）に行います。

7. フェーズ3：OpenLDAP（ユーザーディレクトリ）

目的：IdP が「認証」と「ユーザー ID 取得」に使うユーザーの元データを、WSL2 上の OpenLDAP に用意する。本構成で PLM へ渡すのは個人番号1つのため、LDAP も個人番号（uid）を中心とした最小構成でよい。

7.1 ディレクトリ設計

```
dc=plm-lab,dc=local
├── ou=people
│   ├── uid=90001
│   ├── uid=90002
│   └── uid=idp-reader
└── ou=groups
```

項目	値	役割
ベース DN	dc=plm-lab,dc=local	ディレクトリの頂点
ユーザー OU	ou=people,dc=plm-lab,dc=local	利用者を格納
管理者	cn=admin,dc=plm-lab,dc=local	slapd 管理者（構成変更用）
テストユーザー	uid=90001 / uid=90002	個人番号を uid に格納。SSO ログイン検証用
検索バインド	uid=idp-reader,ou=people,dc=plm-lab,dc=local	IdP が LDAP を検索するための読取専用アカウント
グループ OU	ou=groups,dc=plm-lab,dc=local	将来用。本構成では未使用（認可は PLM の DB 照合）

設計方針：uidに個人番号（5桁数字）を格納し、これを後段で NameID にマッピングして PLM へ渡す。氏名等はスキーマ上必須の cn/sn を形式的に付与するだけで、SAML では運ばない。

7.2 slapd の導入と初期化

```
sudo apt update
sudo apt -y install slapd ldap-utils
```

インストール時に管理者パスワードを尋ねられます。ベース DN を設計どおりにするため、続けて再構成します。

```
sudo dpkg-reconfigure slapd
```

プロンプトでの回答：

- OpenLDAP サーバの設定を省略しますか → **いいえ**
- DNS ドメイン名 → plm-lab.local （→ ベース DN dc=plm-lab,dc=local になる）
- 組織名 → PLM-Lab
- 管理者パスワード → 任意（控えておく）
- データベースを削除しますか（purge 時） → **いいえ**
- 古いデータベースを移動しますか → **はい**

初期化の確認：

```
systemctl status slapd          # active (running) を確認
ldapsearch -x -b "dc=plm-lab,dc=local" -s base # ベースエントリが返る
```

7.3 OU (people / groups) の作成

LDIF ファイルを作成・投入する作業ディレクトリは任意でよいですが、整理のためフェーズ3用のディレクトリを作ってそこで作業することを推奨します（以降の users.ldif/binduser.ldif も同じ場所に置く）。

```
mkdir -p ~/lab-ldap && cd ~/lab-ldap
```

ldapadd は同じディレクトリで実行すればファイル名だけで指定できます（別ディレクトリからなら -f ~/lab-ldap/ou.ldif のようにパス付きで指定）。LDIF はパスワードハッシュを含むため、作業後は chmod 600 *.ldif で保護するか削除してください。

vi ou.ldif で以下を作成：

```
dn: ou=people,dc=plm-lab,dc=local
objectClass: organizationalUnit
ou: people
```

```
dn: ou=groups,dc=plm-lab,dc=local
objectClass: organizationalUnit
ou: groups
```

登録（管理者でバインド。-W でパスワードを対話入力）：

```
ldapadd -x -D "cn=admin,dc=plm-lab,dc=local" -W -f ou.ldif
```

7.4 テストユーザーと検索バインドアカウントの作成

パスワードは平文で書かず、ハッシュを生成して使います。ユーザーごとに実行し、出力の {SSHA}... を控えます。

```
slappasswd          # テストユーザー 90001用のパスワードを入力→{SSHA}... を控える
```

vi users.ldif で以下を作成（userPassword は上で得た各ハッシュに置換）：

```
dn: uid=90001,ou=people,dc=plm-lab,dc=local
objectClass: inetOrgPerson
uid: 90001
cn: Test User 90001
sn: "90001"
userPassword: {SSHA}xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

```
dn: uid=90002,ou=people,dc=plm-lab,dc=local
objectClass: inetOrgPerson
uid: 90002
cn: Test User 90002
sn: "90002"
userPassword: {SSHA}yyyyyyyyyyyyyyyyyyyyyyyyyy
```

vi binduser.ldif で検索バインドアカウントを作成（slappasswd で別途ハッシュ生成）：

```
dn: uid=idp-reader,ou=people,dc=plm-lab,dc=local
objectClass: inetOrgPerson
uid: idp-reader
cn: IdP Reader
sn: Reader
userPassword: {SSHA}zzzzzzzzzzzzzzzzzzzzzzzzzzzz
```

登録：

```
ldapadd -x -D "cn=admin,dc=plm-lab,dc=local" -W -f users.ldif
ldapadd -x -D "cn=admin,dc=plm-lab,dc=local" -W -f binduser.ldif
```

Ubuntu の slapd 既定 ACL では、認証済みバインドでのユーザー検索が可能です。IdP は「idp-reader でバインド → 対象ユーザーを検索 → そのユーザー DN で再バインドしてパスワード検証」という動作をするため、追加の ACL 変更は不要です。

7.5 動作確認

1) 管理者でユーザー検索

```
ldapsearch -x -D "cn=admin,dc=plm-lab,dc=local" -W -b "ou=people,dc=plm-lab,dc=local" "(uid=90001)"
```

2) テストユーザー自身でバインド（=認証テスト。IdP のユーザー認証を模擬）

```
ldapwhoami -x -D "uid=90001,ou=people,dc=plm-lab,dc=local" -W
```

3) 検索バインドで対象を引く（=IdP の検索を模擬）

```
ldapsearch -x -D "uid=idp-reader,ou=people,dc=plm-lab,dc=local" -W \
-b "ou=people,dc=plm-lab,dc=local" "(uid=90001)" uid cn
```

#	確認内容	期待結果
1	slapd 稼働	systemctl status slapd が active (running)
2	ベース DN	ldapsearch -x -b dc=plm-lab,dc=local -s base が成功

#	確認内容	期待結果
3	ユーザー検索	uid=90001 のエントリが返る
4	認証（自己バインド）	ldapwhoami が dn:uid=90001,... を返す（パスワード一致）
5	検索バインド	idp-reader で uid=90001 を取得できる

7.6 フェーズ5（IdP）への引き継ぎ値

フェーズ5でIdPのLDAPコネクタに設定する値です。

項目	値
LDAP URL	ldap://localhost:389
ユーザー検索ベース	ou=people,dc=plm-lab,dc=local
検索バインド DN	uid=idp-reader,ou=people,dc=plm-lab,dc=local
ユーザー検索フィルタ	(uid=\$resolvedUsername)
NameID に載せる属性	uid（＝個人番号）

8. フェーズ4：OpenJDK 17 + Apache Tomcat 10.1

目的： Shibboleth IdP 5 を動かす土台となる Java 17 と Servlet コンテナ Tomcat 10.1 を用意する。IdP 5 は **Java 17 と Tomcat 10.1（Jakarta 名前空間）** を要件とするため、バージョンを厳密に合わせる（Tomcat 9 系・11 系は不可）。

8.1 OpenJDK 17 の導入

```
sudo apt -y install openjdk-17-jdk
java -version          # "17.x" 系であることを確認
readlink -f $(which java) # JAVA_HOME の確認用（例: /usr/lib/jvm/java-17-openjdk-amd64/bin/java）
```

JAVA_HOME は上記 readlink の bin/java を除いたディレクトリ（例 /usr/lib/jvm/java-17-openjdk-amd64）。8.3 のサービス定義で使います。

8.2 Tomcat 10.1 の手動インストール

「できる限り手作業」の方針に沿い、tarball から導入します。バージョンは本書執筆時点の最新 10.1.x（例: **10.1.56**）。最新の 10.1.x は Tomcat 10.1 のダウンロードページで確認してください（オフライン環境では付録 E の方針で tarball を持ち込む）。

サービス用ユーザーと配置先を作成：

```
sudo useradd -r -m -U -d /opt/tomcat -s /usr/sbin/nologin tomcat
```

取得・展開・権限設定（VER を最新 10.1.x に置換）：

```
cd /tmp
VER=10.1.56
wget https://dlcdn.apache.org/tomcat/tomcat-10/v${VER}/bin/apache-tomcat-${VER}.tar.gz
sudo tar xzf apache-tomcat-${VER}.tar.gz -C /opt/tomcat --strip-components=1
sudo chown -R tomcat: /opt/tomcat
# 展開結果の確認（グロブを使わずディレクトリを見る）
sudo ls /opt/tomcat/bin
```



```
# 実行権付与 (グロブ *.sh は root のシェルで展開させるため sudo bash -c でくくる)
sudo bash -c 'chmod +x /opt/tomcat/bin/*.sh'
```

注記（正常な挙動）：sudo chown 後は /opt/tomcat が tomcat 所有（drwxr-x---）になるため、一般ユーザーからの ls /opt/tomcat は Permission denied になります。これは正常で、中身を見るときは sudo ls -l /opt/tomcat を使います。また sudo chmod +x /opt/tomcat/bin/*.sh と直接書くと、*.sh の展開を行うのは非 root のログインシェルのため一致せず No such file になります。上記のように sudo bash -c '...' でくくると root がグロブ展開するため正しく効きます。

学習用途では、既定同梱の docs/examples/manager/host-manager を削除して IdP 専用にしておくとも安全です（任意）。

```
sudo rm -rf /opt/tomcat/webapps/{docs,examples,manager,host-manager}
```

8.3 systemd サービス化

sudo vi /etc/systemd/system/tomcat.service で以下を作成（JAVA_HOME は 8.1 の確認結果に合わせる）：

```
[Unit]
Description=Apache Tomcat 10.1
After=network.target

[Service]
Type=forking
User=tomcat
Group=tomcat
Environment="JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64"
Environment="CATALINA_HOME=/opt/tomcat"
Environment="CATALINA_BASE=/opt/tomcat"
Environment="CATALINA_PID=/opt/tomcat/temp/tomcat.pid"
Environment="CATALINA_OPTS=-Xms512M -Xmx1024M -server"
ExecStart=/opt/tomcat/bin/startup.sh
ExecStop=/opt/tomcat/bin/shutdown.sh
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

有効化・起動：

```
sudo systemctl daemon-reload
sudo systemctl enable --now tomcat
sudo systemctl status tomcat      # active (running) を確認
```

8.4 動作確認

```
curl -I http://localhost:8080/      # HTTP/1.1 200 が返る
ss -ltnp | grep 8080               # Tomcat が 8080 で待受
java -version                       # 17 系
```

#	確認内容	期待結果
1	Java	java -version が 17 系
2	Tomcat 稼働	systemctl status tomcat が active (running)
3	HTTP 応答	curl -I http://localhost:8

#	確認内容	期待結果
		080/ が 200
4	待受ポート	8080 で listen

Tomcat の 8080 は当面 **localhost 用** でよい（ブラウザからの HTTPS 到達はフェーズ 6 の Apache 前段 8443 が担う）。IdP はこの Tomcat 上に /idp として配置します（フェーズ 5）。

8.5 フェーズ 5（IdP）への引き継ぎ値

項目	値
JAVA_HOME	/usr/lib/jvm/java-17-openjdk-amd64（8.1 の確認値）
CATALINA_HOME / BASE	/opt/tomcat
Tomcat コネクタ	http://localhost:8080（TLS は前段 Apache が担当）
IdP 配置先	Tomcat（/idp。IdP インストーラが war を生成）

9. フェーズ 5：Shibboleth IdP 5

目的：フェーズ 4 の Tomcat 上に Shibboleth IdP 5 を導入し、フェーズ 3 の OpenLDAP を認証元として「個人番号（uid）でログインできる IdP」を立ち上げる。SAML 署名・暗号化証明書（(b)）はインストーラが自動生成する。個人番号を NameID／属性のどちらで SP へ渡すかの最終設定は、PLM の受け口の確認結果を踏まえフェーズ 9・10 で確定する（本フェーズでは LDAP 認証の成立までを目標とする）。

重要（要件の再掲）：IdP 5 は Java 17・Tomcat 10.1（Jakarta 名前空間）を要件とする（フェーズ 4 で用意済み）。Java 17 は Nashorn（JavaScript エンジン）を同梱しないため、スクリプト属性を使う場合は別途 Nashorn プラグインが必要（本構成の最小設定では未使用）。

9.1 IdP 5 のダウンロードと展開

Tomcat をいったん停止してから作業します。**バージョンは固定せず、latest5/ に実在する版を確認してから取得**します（latest5/ には最新版のみ置かれ、古い版番号を指定すると 404 になるため）。

```
sudo systemctl stop tomcat
cd /usr/local/src
```

```
# 1) latest5/ に実在するファイル名（＝現行版）を確認
curl -sL https://shibboleth.net/downloads/identity-provider/latest5/ \
  | grep -o 'shibboleth-identity-provider-[0-9.]*\.tar\.gz' | sort -u
# 例: shibboleth-identity-provider-5.2.3.tar.gz
```

```
# 2) 表示された版を VER に設定し、-L（リダイレクト追従）で取得
VER=5.2.3 # ← 手順1で表示された実在版に置換
sudo curl -L -O "https://shibboleth.net/downloads/identity-provider/latest5/shibboleth-identity-provider-${VER}.tar.gz"
```

```
# 3) 取得物を検証（数十MB・gzip であること）
ls -l shibboleth-identity-provider-${VER}.tar.gz # 数千万バイト（例 43MB 前後）
```

```
file shibboleth-identity-provider-${VER}.tar.gz      # "gzip compressed data"
```

4) 展開

```
sudo tar xzf shibboleth-identity-provider-${VER}.tar.gz
cd shibboleth-identity-provider-${VER}
```

⚠ curl -O (-L なし) や存在しない版番号だと、196 バイト程度の HTML (404 やリダイレクト案内) が保存され、gzip: not in gzip format で展開に失敗します。手順3の ls -l (サイズ) と file (gzip compressed data) で必ず本体か確認してください。

9.2 インストーラの実行

```
sudo ./bin/install.sh
```

対話プロンプトでの回答 ([] は既定値。入力後 Enter) :

プロンプト	入力値
Installation Directory	/opt/shibboleth-idp
Host Name	idp.plm-lab.local
SAML EntityID	https://idp.plm-lab.local/idp/shibboleth
Attribute Scope	plm-lab.local
Keystore / Sealer パスワード	任意 (控える)

完了すると idp.home = /opt/shibboleth-idp に設定ツリーが作られ、SAML 用の署名・暗号鍵 ((b)) と自己署名証明書が credentials/ に自動生成されます。この (b) は CA 信頼もホスト名一致も不要で、フェーズ9のメタデータ交換で相手 (SP) に登録します。

インストール確認 (コンテナ外での自己診断) :

```
sudo JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64 /opt/shibboleth-idp/bin/status.sh
```

9.3 Tomcat への配置 (context フラグメント・JSTL・権限)

(1) context フラグメント: Tomcat に IdP の war の位置を教えます。

```
sudo mkdir -p /opt/tomcat/conf/Catalina/localhost
echo '<Context docBase="/opt/shibboleth-idp/war/idp.war" privileged="true" antiResourceLocking="false"
swallowOutput="true" />' \
| sudo tee /opt/tomcat/conf/Catalina/localhost/idp.xml
```

(2) JSTL の追加: IdP 5 のビュー (ログイン/エラー画面) に JSTL が必要です。追加後にビルドします。

```
cd /opt/shibboleth-idp
sudo mkdir -p edit-webapp/WEB-INF/lib
sudo curl -O --output-dir edit-webapp/WEB-INF/lib/ \
https://repo.maven.apache.org/maven2/jakarta/servlet/jsp/jstl/jakarta.servlet.jsp.jstl-api/3.0.0/jakarta.servlet.jsp.jstl-api-3.0.0.jar
sudo JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64 /opt/shibboleth-idp/bin/build.sh
```

(3) 権限: Tomcat (tomcat ユーザー) が読めるように調整します。

```
sudo chown -R tomcat /opt/shibboleth-idp/{logs,metadata}
sudo chgrp -R tomcat /opt/shibboleth-idp/{credentials,conf}
sudo chmod -R g+r /opt/shibboleth-idp/conf
sudo chmod 750 /opt/shibboleth-idp/credentials
# 直後は credentials が 750 で一般ユーザーから中が見えないため、グロブは root で展開させる
sudo bash -c 'chmod 640 /opt/shibboleth-idp/credentials/*'
```

sudo chmod 640 /opt/shibboleth-idp/credentials/* と直接書くと、*を展開するのは非 root のシェルで、chmod 750 直後の credentials を読めず No such file になります。sudo bash -c '...' でくると root がグロブ展開するため正しく効きます（フェーズ 4 §8.2 と同じ注意点）。この 640 化により tomcat グループが署名・暗号鍵(b)を読めるようになります（600 のままだと IdP 起動時に鍵読み込みで失敗します）。

9.4 LDAP 認証の設定（フェーズ 3 の値を使用）

sudo vi /opt/shibboleth-idp/conf/ldap.properties を編集し、§7.6 の値を設定します（既存の該当行を書き換え、二重定義を避ける）。

```
idp.authn.LDAP.authenticator      = bindSearchAuthenticator
idp.authn.LDAP.ldapURL            = ldap://localhost:389
idp.authn.LDAP.useStartTLS        = false
idp.authn.LDAP.useSSL             = false
idp.authn.LDAP.baseDN             = ou=people,dc=plm-lab,dc=local
idp.authn.LDAP.userFilter         = (uid={user})
idp.authn.LDAP.bindDN             = uid=idp-reader,ou=people,dc=plm-lab,dc=local
idp.authn.LDAP.dnFormat           = uid=%s,ou=people,dc=plm-lab,dc=local
idp.authn.LDAP.returnAttributes  = uid
```

注意点（実機で判明）：- dnFormat の既定は ...dc=example,dc=org のままなので、上記のとおり plm-lab.local に直す（bindSearchAuthenticator では通常参照されないが、誤値を残さない）。- useStartTLS の既定は true のことが多い。false への変更を忘れない。TLS 無効のため trustCertificates/trustStore の行はコメントアウトしてよい。- bindSearchAuthenticator は「idp-reader で検索 → 見つかったユーザー DN で再バインドしてパスワード検証」という、フェーズ 3 §7.5 で確認した動作。組織展開時は LDAPS/StartTLS を推奨。

バインドパスワードは secrets.properties に集約します。 ldap.properties に bindDNCredential を書くと、インストーラが生成した credentials/secrets.properties の定義と重複し、Duplicate properties ... bindDNCredential の WARN が出ます。パスワードは次のように **secrets 側に一本化**します。

```
# secrets.properties の既定 (myServicePassword) を idp-reader の平文パスワードに書き換え
sudo vi /opt/shibboleth-idp/credentials/secrets.properties
# idp.authn.LDAP.bindDNCredential = <idp-reader の平文パスワード>
# (その下の idp.attribute.resolver.LDAP.bindDNCredential = %{...} は参照なので触らない)

# ldap.properties 側に bindDNCredential を書いた場合はコメントアウト（未記入なら不要）
```

bindDNCredential は **平文パスワード**（LDAP サーバへバインドする側が送る値）。フェーズ 3 で idp-reader に設定した平文をそのまま記述します（{SSHA} ハッシュではありません）。不安なら ldapwhoami -x -D "uid=idp-reader,ou=people,dc=plm-lab,dc=local" -w '<平文>' で検証できます。

9.5 個人番号 (uid) の属性解決 (Phase 9/10 への準備)

conf/attribute-resolver.xml に、LDAP から uid を取得する DataConnector と uid の AttributeDefinition を定義します（IdP 5 同梱のテンプレートの LDAP コネクタを有効化し、uid を対象にするのが最小構成）。

本フェーズでの扱い：個人番号を NameID として送るか属性として送るかは、PLM の受け口（REMOTE_USER で受ける等）の確認結果に依存します。そのため、**属性解放ポリシー (attribute-filter)** と NameID 生成の最終設定はフェーズ 9・10 で確定します。本フェーズで

は「uid を解決できる状態」までを用意します。具体的な XML は同梱テンプレートと Shibboleth 公式ドキュメント (attribute-resolver / saml-nameid) に従ってください。

9.6 起動と動作確認

```
sudo systemctl start tomcat
```

IdP のステータス (当面は Tomcat の 8080 経由。ブラウザ HTTPS はフェーズ 6 で)

```
curl -s http://localhost:8080/idp/profile/status ; echo
```

```
sudo tail -n 50 /opt/shibboleth-idp/logs/idp-process.log
```

#	確認内容	期待結果
1	IdP 配置	/opt/shibboleth-idp/war/idp.war が存在、idp.xml を配置
2	Tomcat 起動	systemctl status tomcat が active、catalina.out に IdP 起動ログ
3	ステータス	http://localhost:8080/idp/profile/status が応答 (idp-process.log にエラーなし)
4	メタデータ	http://localhost:8080/idp/shibboleth が IdP メタデータ (XML) を返す
5	LDAP 認証	idp-process.log に LDAP 接続エラーが出ない (実ログインは SP 接続後のフェーズ 9 で確認)

起動失敗時は `sudo tail -n 100 /opt/tomcat/logs/catalina.out` と `/opt/shibboleth-idp/logs/idp-process.log` を確認。多くは JSTL 未追加、context フラグメントのパス誤り、`ldap.properties` の値誤り、権限不足が原因。

9.7 フェーズ 6・9 への引き継ぎ値

項目	値
IdP entityID	https://idp.plm-lab.local/idp/shibboleth
IdP メタデータ URL	http://localhost:8080/idp/shibboleth (前段 Apache 経由では https://idp.plm-lab.local:8443/idp/shibboleth)
バックエンド	Tomcat http://localhost:8080/idp (TLS はフェーズ 6 の Apache が 8443 で終端)
SAML 署名・暗号化証明書 (b)	/opt/shibboleth-idp/credentials/ (フェーズ 9 で SP に登録)

10. フェーズ 6：Apache HTTPD (IdP 前段のリバースプロキシ・8443/TLS)

目的：IdP (Tomcat 8080、HTTP) の前段に Apache HTTPD を置き、フェーズ 2 で作成した **idp サーバ証明書 (a)** で https://idp.plm-lab.local:8443/ を TLS 終端して内部の Tomcat へ中継する。これにより、初めて **ブラウザから HTTPS で IdP に到達**でき、以降フェーズ 7・8 の SP と接続する準備が整う。

構成の再確認：ブラウザ → (HTTPS 8443) → Apache (TLS 終端) → (HTTP 8080) → Tomcat/IdP。8443 にするのは、同一ホストで SP (IIS, 443) と 443 が衝突しないようにするため

(\$6.2)。段階アプローチの (A) では、ゲスト Windows のブラウザから WSL2 の 8443 へ localhost 経由で到達する。

10.1 Apache HTTPD と必要モジュールの導入

```
sudo apt -y install apache2
sudo a2enmod ssl proxy proxy_http headers
```

10.2 証明書の配置

フェーズ 2 で ~/lab-ca に作成した idp 証明書 (a) と CA を、Apache が読める場所へ配置します。

```
sudo mkdir -p /etc/apache2/ssl
sudo cp ~/lab-ca/idp.crt /etc/apache2/ssl/
sudo cp ~/lab-ca/idp.key /etc/apache2/ssl/
sudo cp ~/lab-ca/rootCA.crt /etc/apache2/ssl/
sudo chmod 600 /etc/apache2/ssl/idp.key # 秘密鍵 (Apache は起動時に root で読む)
```

10.3 Tomcat コネクタをプロキシ対応にする

IdP が「外部からは https://idp.plm-lab.local:8443 で見えている」と認識できるように、Tomcat 8080 コネクタにプロキシ情報を付与します。sudo vi /opt/tomcat/conf/server.xml で 8080 の <Connector> を次のように変更します。

```
<Connector port="8080" protocol="HTTP/1.1"
  connectionTimeout="20000"
  proxyName="idp.plm-lab.local" proxyPort="8443"
  scheme="https" secure="true"
  redirectPort="8443" />
```

これにより IdP はリクエストを https://idp.plm-lab.local:8443/... として扱い、生成する URL (リダイレクト先等) が正しく 8443 の HTTPS になります。変更後は Tomcat を再起動します (sudo systemctl restart tomcat)。

10.4 8443 の VirtualHost を作成

まず 8443 を待ち受けに追加します (ports.conf に Listen 8443 を追記。重複追加を避ける)。

```
grep -q "Listen 8443" /etc/apache2/ports.conf || echo "Listen 8443" | sudo tee -a /etc/apache2/ports.conf
```

重要 (mirrored 環境では必須)： mirrored モードでは WSL2 が Windows のネットワークを共有するため、**Windows 側の IIS が使う 80/443 と、Apache 既定の Listen 80 / Listen 443 が衝突し**、Apache が (98)Address already in use で起動できなくなります。Apache は IdP 前段として **8443 専用** でよいので、ports.conf の Listen 80 と Listen 443 を **コメントアウト** し、80 番の既定サイトも無効化します。

```
sudo sed -i 's/^Listen 80/#Listen 80/; s/^\(s*\)Listen 443/\1#Listen 443/' /etc/apache2/ports.conf
sudo a2dissite 000-default 2>/dev/null || true
```

(NAT の (A) 段階では 80 の衝突は起きないが、mirrored ((B)) へ移行すると顕在化する。フェーズ 6 を mirrored 前提で行う場合は本設定を必ず実施。)

sudo vi /etc/apache2/sites-available/idp-8443.conf で以下を作成します。

```
<VirtualHost *:8443>
  ServerName idp.plm-lab.local:8443
```

```

SSLEngine on
SSLCertificateFile      /etc/apache2/ssl/idp.crt
SSLCertificateKeyFile   /etc/apache2/ssl/idp.key
SSLCertificateChainFile /etc/apache2/ssl/rootCA.crt

ProxyPreserveHost On
RequestHeader set X-Forwarded-Proto "https"

ProxyPass      /idp http://localhost:8080/idp
ProxyPassReverse /idp http://localhost:8080/idp
</VirtualHost>

```

サイトを有効化し、構文チェックして再起動します。

```

sudo a2ensite idp-8443
sudo apache2ctl configtest      # "Syntax OK"
sudo systemctl restart apache2

```

10.5 動作確認

```

# WSL2 内から (CA は §6.5 で update-ca-certificates 済みなので -k 不要のはず)
curl -I https://idp.plm-lab.local:8443/idp/profile/status
# 証明書の検証状況を見たい場合
curl -v https://idp.plm-lab.local:8443/idp/profile/status 2>&1 | grep -i 'SSL certificate\|subject\|issuer\|200'

```

次に、**ゲスト Windows のブラウザ**（段階 (A)）で以下を開きます。

- `https://idp.plm-lab.local:8443/idp/profile/status`

#	確認内容	期待結果
1	Apache 構文	apache2ctl configtest が Syntax OK
2	Apache 稼働	systemctl status apache2 が active
3	HTTPS 到達	curl -I https://idp.plm-lab.local:8443/idp/profile/status が 200
4	証明書検証	curl が証明書エラーを出さない (rootCA 信頼済み)
5	ブラウザ	ゲスト Windows のブラウザで鍵マーク (証明書エラーなし)、ステータス表示

つまづきやすい点：ブラウザで証明書警告が出る場合は、フェーズ 2 の rootCA が Windows の「信頼されたルート証明機関」に入っているか (§6.5) を確認。502/503 が出る場合は Tomcat/IdP が 8080 で起動しているか (curl -I http://localhost:8080/idp/profile/status)、a2enmod proxy proxy_http が有効かを確認。リダイレクトが http:// や別ポートになる場合は 10.3 の Tomcat コネクタ設定を再確認。**ブラウザで「接続が拒否されました」になる場合は、WSL2 の localhost 転送が効いていない可能性が高い。** §6.3 の実機知見に従い Test-Net Connection で切り分け、networkingMode=mirrored へ移行する (curl は WSL2 内で通るのにブラウザだけ繋がらない、という症状が典型)。

10.6 フェーズ 9 への引き継ぎ

項目	値
IdP 外部 URL	https://idp.plm-lab.local:8443/idp/
IdP メタデータ (外部)	https://idp.plm-lab.local:8443/idp/shibboleth

項目	値
SSO エンドポイント	https://idp.plm-lab.local:8443/idp/profile/SAML2/Redirect/SSO 等
インストール時に生成された metadata/idp-metadata.xml の各エンドポイント Location は、entityID 由来で ポートなし (443) になっている場合があります。フェーズ9で SP に渡すメタデータでは、エンドポイントを :8443 に合わせる（編集または再生成する）必要があります。この調整はフェーズ9で扱います。	

11. フェーズ7：IIS（SPの保護対象サイト・443/TLS）

目的：Windows 仮想マシンに IIS を導入し、保護対象となる PLM 相当のサイト（認証後に個人番号＝REMOTE_USER を表示する確認ページ）を 443/HTTPS で用意する。フェーズ2の **sp 証明書 (a)** を Windows 側で初めて使う。フェーズ8で Shibboleth SP（ISAPI）を組み込む土台となる。

ネットワーク前提：フェーズ6で mirrored モードに移行済みのため、ゲスト Windows のブラウザは localhost 経由で SP(443, IIS=Windows ネイティブ) と IdP(8443, WSL2) の双方に到達できる。IIS はゲスト Windows 上で動くので、sp.plm-lab.local-127.0.0.1 で素直に届く。

11.1 IIS の導入（Windows の機能）

フェーズ8の Shibboleth SP は **ISAPI フィルタ**として動くため、ISAPI 拡張／フィルタを含めて有効化します。確認ページ用に**古典 ASP**も入れます。ゲスト Windows の**管理者 PowerShell**で：

```
Enable-WindowsOptionalFeature -Online -All -FeatureName `
    IIS-WebServerRole, IIS-WebServer, IIS-CommonHttpFeatures, IIS-StaticContent, `
    IIS-DefaultDocument, IIS-ISAPIExtensions, IIS-ISAPIFilter, IIS-ASP, `
    IIS-RequestFiltering, IIS-WebServerManagementTools, IIS-ManagementConsole
```

GUIで行う場合：「コントロール パネル → プログラム → Windows の機能の有効化または無効化 → インターネット インフォメーション サービス」で、特に「World Wide Web サービス → アプリケーション開発機能 → **ISAPI 拡張機能／ISAPI フィルター／ASP**」と「Web 管理ツール → **IIS 管理コンソール**」を有効化。

確認：ブラウザで http://localhost/ に IIS の既定ページが表示される。

11.2 hosts に sp.plm-lab.local を追加

```
Add-Content -Path C:\Windows\System32\drivers\etc\hosts -Value "`n127.0.0.1`tsps.plm-lab.local"
ping -n 1 sp.plm-lab.local # 127.0.0.1 に解決されること
```

11.3 sp 証明書 (a) を Windows に取り込む

フェーズ2 §6.5 で作成した sp.pfx（WSL2 の ~/lab-ca/sp.pfx）を、Windows の証明書ストア（ローカルコンピュータ\個人）へ取り込みます。\\wsl\$ 経由で参照します。

```
Import-PfxCertificate -FilePath "\\wsl$\Ubuntu-24.04\home\infodba\lab-ca\sp.pfx" `
    -CertStoreLocation Cert:\LocalMachine\My `
    -Password (Read-Host "PFXのエクスポートパスワード" -AsSecureString)
```

パスワードはフェーズ2で openssl pkcs12 -export の際に設定したもの。rootCA はフェーズ2 §6.5 で「信頼されたルート証明機関」に登録済みのため、ブラウザは sp 証明書を信頼できます。

11.4 既定サイトに 443/HTTPS バインドを追加

```
Import-Module WebAdministration
$cert = Get-ChildItem Cert:\LocalMachine\My |
    Where-Object { $_.Subject -like "*CN=sp.plm-lab.local*" } | Select-Object -First 1
New-WebBinding -Name "Default Web Site" -Protocol https -Port 443
New-Item -Path "IIS:\SslBindings\0.0.0.0!443" -Value $cert
```

GUIで行う場合：IIS マネージャー → 「Default Web Site」 → 右側「バインド」 → 「追加」
→ 種類 https / ポート 443 / SSL 証明書に sp.plm-lab.local を選択。

11.5 確認用ページ (REMOTE_USER 表示)

C:\inetpub\wwwroot\whoami.asp を作成します（メモ帳を管理者で開いて保存）。

```
<%@ Language="VBScript" %>
<html><body>
<h2>認証確認ページ</h2>
REMOTE_USER = [<%= Request.ServerVariables("REMOTE_USER") %>]<br>
AUTH_TYPE = [<%= Request.ServerVariables("AUTH_TYPE") %>]
</body></html>
```

この段階では SP 未導入のため REMOTE_USER は空 ([]) で表示されます。これは正常で、フェーズ 8～9 で SAML 認証が通ると、ここに**個人番号**が入ります。認証連携の成否をこのページで目視確認します。

補足 1 (ファイル作成権限・OS 差異)：C:\inetpub\wwwroot への書き込みは、**昇格したプロセス**で行うのが確実（例：管理者 PowerShell の Set-Content、または「管理者として実行」したメモ帳）。ビルトイン Administrator の挙動は OS で異なり、**Windows Server 2016** は既定で管理者承認モードが無効のためエクスプローラーの「新規作成」で wwwroot に直接書けるが、**Windows 11 クライアント**は承認モードが効きエクスプローラーが標準トークンで動くため書けない（UAC スライダーを最下段にしても変わらない）。承認モードの無効化（FilterAdministratorToken 等）はセキュリティ低下のため非推奨で、昇格プロセスでの作成を推奨。組織の検証環境（Server 系）ではこの制限自体が起きにくい。

補足 2 (文字コード)：日本語を含む古典 ASP を**英語ロケール**の IIS で表示すると文字化けすることがある。その場合はページ先頭で Response.CodePage=65001 / Response.Charset="utf-8" を指定し、ファイルを **BOM 無し UTF-8** で保存する。確認用途では見出しを英語表記（例：Authentication Test）にして回避してもよい。

11.6 動作確認

ゲスト Windows のブラウザで確認します。

#	確認内容	期待結果
1	IIS 既定ページ	http://localhost/ が表示される
2	HTTPS バインド	https://sp.plm-lab.local/ が 鍵マーク （証明書エラーなし）で表示
3	ASP 動作	https://sp.plm-lab.local/whoami.asp が表示される
4	REMOTE_USER	同ページで REMOTE_USER = []（空。SP 未導入のため正常）

証明書警告が出る場合は、rootCA が「信頼されたルート証明機関」にあるか (§6.5) を確認。whoami.asp が表示されず 500 等になる場合は、古典 ASP 機能 (§11.1 の IIS-ASP) が有効か、ファイルの拡張子が .asp かを確認。

11.7 フェーズ 8 への引き継ぎ値

項目	値
SP entityID	https://sp.plm-lab.local/shibboleth
保護対象 (予定)	/whoami.asp (フェーズ 8 で SP が保護し、REMOTE_USER に個人番号が入る)
SP 方式	ISAPI (IIS-ISAPIFilter 有効化済み)
サイト	Default Web Site (443/HTTPS、sp 証明書 (a))

12. フェーズ 8 : Shibboleth SP (IIS ネイティブモジュール・サイト全体保護)

目的：Windows 仮想マシンに Shibboleth SP 3 を導入し、IIS と連携させて Default Web Site 全体を SAML 保護する。SP の SAML 署名・暗号鍵 (b) を生成し、SP メタデータを取得できる状態にする。実際の SSO 成立 (IdP へのリダイレクト～アサーション受領) は、IdP/SP のメタデータを相互登録するフェーズ 9 以降で完成する。

前提：フェーズ 7 で IIS (ISAPI 有効) ・ 443/HTTPS ・ 確認ページ whoami.asp を用意済み。SP は shibd デーモンと IIS ネイティブモジュールの 2 つで構成される。

12.1 SP インストーラの入手と導入

- 公式サイト <https://shibboleth.net/downloads/service-provider/latest/> の **win64/** から最新の **msi** を入手 (バージョンは固定せず最新を使用)。
- MSI を実行し、既定のまま進める。要点：
 - インストール先は既定の C:\opt\shibboleth-sp。
 - 「Configure IIS7 module? (IIS サポートの構成)」にチェック (IIS ネイティブモジュールを自動構成)。
 - 完了後、再起動を求められるので再起動する。

配置の注記 (設計上の推奨)：インストール先はデフォルトの C:\opt\shibboleth-sp を推奨。C:\inetpub 配下は、SP の秘密鍵・設定が Web 公開領域に入り漏洩リスクがあるため不可。組織のポリシーで配置を集約する場合も、空白を含まないパス (例 C:\app\shibboleth-sp) にとどめ、shibboleth2.xml 内のパス・keygen 出力先・ログ/鍵のパスを一斉に読み替える必要がある。学習・検証ではデフォルトが最も安全で、ドキュメントとの整合も良い。

12.2 インストールの確認

- サービス：「サービス」管理コンソールで **Shibboleth 3 Daemon** が「実行中／自動／Local System」であること。
- IIS (SP3 のネイティブモジュール方式)：IIS マネージャーでサーバーを選択 → 「モジュール」に **ShibNative/ShibNative32** (C:\opt\shibboleth-sp\lib64\shibboleth\iis7_shib.dll 等、Native/Local) があること。※ SP3 はネイティブモジュール方式のため、旧 ISAPI 方式の *.sso ハンドラーマッピングは表示されないのが正常。下のステータス確認が通れば実質確認済み。
- ステータス (必ず localhost で、大文字小文字を区別)：ブラウザで <https://localhost/Shibboleth.sso/Status> を開き、末尾に <Status><OK/></Status> が返ること。

動かない場合は shibd の設定チェック：管理者コマンドプロンプトで C:\opt\shibboleth-sp\sbin\shibd.exe -check -config C:\opt\shibboleth-sp\etc\shibboleth\shibboleth2.xml → overall configuration is loadable... なら設定は読み込み可能。ログは C:\opt\shibboleth-sp\var\log\shibboleth\shibd.log。

12.3 shibboleth2.xml の編集

C:\opt\shibboleth-sp\etc\shibboleth\shibboleth2.xml を編集します（まず shibboleth2.xml.orig にバックアップ。タイプミスが最大の事故要因なので慎重に）。変更点は次の4か所です。

(1) <ISAPI> の <Site> (IIS サイト ID とホスト名の対応。Default Web Site の ID は通常 1)

```
<ISAPI normalizeRequest="true" safeHeaderNames="true">
  <Site id="1" name="sp.plm-lab.local" scheme="https" port="443"/>
</ISAPI>
```

(2) <RequestMapper> の <Host> (サイト全体を保護)

```
<RequestMapper type="Native">
  <RequestMap>
    <Host name="sp.plm-lab.local" authType="shibboleth" requireSession="true"/>
  </RequestMap>
</RequestMapper>
```

requireSession="true" により、このホストへの全アクセスがセッション必須（＝未認証なら IdP へ）になります。これが「サイト全体保護」の設定です。

(3) <ApplicationDefaults> の entityID と REMOTE_USER

```
<ApplicationDefaults entityID="https://sp.plm-lab.local/shibboleth"
  REMOTE_USER="eppn persistent-id targeted-id">
```

REMOTE_USER は「先頭から最初に値のある属性」が採用されます。個人番号 (NameID) をここに載せる最終設定はフェーズ 10で行います (IdP 側の NameID/属性の出し方と対で決めるため)。本フェーズでは既定のままにしておきます。

(4) <SSO> に IdP の entityID を指定

```
<SSO entityID="https://idp.plm-lab.local/idp/shibboleth">
  SAML2
</SSO>
```

IdP のメタデータ (<MetadataProvider>) の登録は、エンドポイントを :8443 に整える必要があるためフェーズ 9で行います。本フェーズでは entityID の指定までにとどめます。

あわせて、Status ハンドラーの acl に必要なら 127.0.0.1 が含まれることを確認します (既定で含まれることが多い)。

12.4 SP 鍵 (b) の生成

SP の SAML 署名・暗号化証明書 (b) を、正しいホスト名・entityID で生成します。管理者コマンドプロンプトで：

```
cd C:\opt\shibboleth-sp\etc\shibboleth
keygen.bat -h sp.plm-lab.local -e https://sp.plm-lab.local/shibboleth -y 10
```

sp-signing-cert.pem / sp-encrypt-cert.pem等が生成されます。これはフェーズ2の(a)とは別物で、**メタデータ交換で IdP に渡す (b)** (CA 信頼・ホスト名一致は不要)。MSI が既定鍵を生成済みの場合もありますが、entityID/ホスト名を正しくするため上記で作り直します。

12.5 反映 (IIS 完全再起動)

<ISAPI> を変更したときは **IIS の完全再起動**が必要です。

```
# shibd 再起動 + IIS 完全再起動
Restart-Service shibd_Default
iisreset
```

12.6 動作確認

#	確認内容	期待結果
1	shibd 稼働	「Shibboleth 3 Daemon」が実行中
2	設定読込	shibd.exe -check が overall configuration is loadable
3	Status	https://localhost/Shibboleth.sso/Status が <Status><OK/></Status>
4	SP メタデータ	https://sp.plm-lab.local/Shibboleth.sso/Metadata が SP メタデータ (XML) を返す。中に sp.example.org が残っていない (すべて sp.plm-lab.local)
5	保護の発火	https://sp.plm-lab.local/whoami.asp にアクセスすると、SP がセッションを要求する (この時点では IdP メタデータ未登録のため SSO は未完了。エラーやメタデータ未検出になるのは想定どおり)

確認4のSPメタデータ (Shibboleth.sso/Metadata) は、フェーズ9で **IdP に登録する SP メタデータ**として使います。ファイルに保存しておくと便利です。

12.7 フェーズ9への引き継ぎ値

項目	値
SP entityID	https://sp.plm-lab.local/shibboleth
SP メタデータ URL	https://sp.plm-lab.local/Shibboleth.sso/Metadata
SP の ACS (想定)	https://sp.plm-lab.local/Shibboleth.sso/SAML2/POST
SP 署名・暗号鍵 (b)	C:\opt\shibboleth-sp\etc\shibboleth\sp-*-cert.pem
次工程	フェーズ9：IdPメタデータ (:8443 に補正) を SP に登録し、SPメタデータを IdP に登録。個人番号の NameID/属性と REMOTE_USER 対応を確定

13. フェーズ9：メタデータ交換 (IdP ↔ SP の相互信頼・初回 SSO 成立)

目的：IdP と SP のメタデータを静的（ファイル）に相互登録し、両者が信頼し合って SAML の往復が成立する状態にする。フェーズ8で出た No MetadataProvider available. を解消し、https://sp.plm-lab.local/whoami.asp への未認証アクセスが IdP のログイン画面（8443）へ遷移 → 個人番号(uid=90001)+パスワードでログイン → SP に戻ってセッション確立、という一連の流れを確認する。

本フェーズのゴールと切り分け：ここでは「SSO の往復が成立し、SP セッションが張れて保護ページに到達できる」ところまでを目標とする。REMOTE_USER に個人番号を載せる最終設定（NameID/属性の解放）はフェーズ10で仕上げる。本フェーズ完了時点では whoami.asp の REMOTE_USER は空でも構わない（セッションが確立し保護ページが開けることが合格条件）。

13.1 IdP メタデータの取得とエンドポイントの :8443 補正

IdP のメタデータは、インストール時に生成された /opt/shibboleth-idp/metadata/idp-metadata.xml にあります。ただし各エンドポイントの Location が **ポートなし (=443)** で書かれているため、このままだと SP はブラウザを https://idp.plm-lab.local/idp/... (443=IIS/SP 側) へ送ってしまい SSO が壊れます。**エンドポイントを :8443 に補正**します（フェーズ6 §10.6 で予告した対応）。

WSL2 で、SP へ渡す用のコピーを作り、ポートを補正します。

```
cd /opt/shibboleth-idp/metadata
sudo cp idp-metadata.xml idp-metadata-for-sp.xml
# エンドポイントの host を :8443 付きに補正 (entityID は変えない)
sudo sed -i 's#https://idp.plm-lab.local/idp/#https://idp.plm-lab.local:8443/idp/#g' idp-metadata-for-sp.xml
# entityID 行だけは元に戻す (識別子はポートなしのまま)
sudo sed -i 's#entityID="https://idp.plm-lab.local:8443/idp/shibboleth"#entityID="https://idp.plm-lab.local/idp/shibboleth"#' idp-metadata-for-sp.xml
grep -o 'Location="[^\"]*"' idp-metadata-for-sp.xml | sort -u # :8443 になっているか確認
```

確認：SingleSignOnService などの Location が https://idp.plm-lab.local:8443/idp/... になり、entityID は https://idp.plm-lab.local/idp/shibboleth (ポートなし) のままであること。

この idp-metadata-for-sp.xml を Windows 側へコピーします (\\wsl\$ 経由)。ゲスト Windows の管理者 PowerShell で：

```
Copy-Item "\\wsl$\Ubuntu-24.04\opt\shibboleth-idp\metadata\idp-metadata-for-sp.xml" `
"C:\opt\shibboleth-sp\etc\shibboleth\idp-metadata.xml"
```

13.2 SP に IdP メタデータを登録

C:\opt\shibboleth-sp\etc\shibboleth\shibboleth2.xml に IdP メタデータの <MetadataProvider> を追加します。**配置場所が重要**で、<MetadataProvider> は <Sessions> の中ではなく、</Sessions> の閉じタグの後・<Errors .../> の下に置きます（<Sessions> 内に置くとスキーマ違反 element 'MetadataProvider' is not allowed for content model ... で shibd が起動しません）。既定ファイルの「Example of locally maintained metadata」コメントの位置がまさにその場所です。

```
</Sessions>

<Errors supportContact="root@localhost"
  helpLocation="/about.html"
  styleSheet="/shibboleth-sp/main.css"/>
```

```
<!-- ここ (Sessions の外・Errors の下) に追加 -->
<MetadataProvider type="XML" validate="true" path="idp-metadata.xml"/>
```

保存後、設定チェックしてから SP を再起動します。ゲスト Windows の管理者コマンドプロンプト／PowerShell で：

```
# 設定チェック (overall configuration is loadable を確認)
C:\opt\shibboleth-sp\sbin\shibd.exe -check
# 再起動
Restart-Service shibd_Default
iisreset
```

これで No MetadataProvider available. は解消します。https://localhost/Shibboleth.sso/Status が引き続き <OK/> であることも確認。

13.3 SP メタデータの取得と IdP への配置

SP のメタデータを取得して IdP 側へ配置します。ブラウザで https://sp.plm-lab.local/Shibboleth.sso/Metadata を開き、**XML を sp-metadata.xml として保存**（フェーズ 8 で保存済みならそれを使用）。これを WSL2 の IdP メタデータ領域へ置きます。

WSL2 で（Windows 上の保存先から /mnt/c/... 経由で取得、またはエディタで貼り付け）：

```
# 例：Windows のダウンロード先から取得する場合
sudo cp /mnt/c/Users/infodba/Downloads/sp-metadata.xml /opt/shibboleth-idp/metadata/sp-metadata.xml
sudo chown tomcat /opt/shibboleth-idp/metadata/sp-metadata.xml
```

13.4 IdP に SP メタデータを登録

/opt/shibboleth-idp/conf/metadata-providers.xml を編集し、既定の <MetadataProvider id="ShibbolethMetadata" xsi:type="ChainingMetadataProvider"> と、それを閉じる </MetadataProvider> の間（**チェーンの内側**）に、SP メタデータの FilesystemMetadataProvider を追加します（コメントアウトされた LocalMetadata 見本の位置が最適）。

```
<MetadataProvider id="LocalSP" xsi:type="FilesystemMetadataProvider"
    metadataFile="%{idp.home}/metadata/sp-metadata.xml"/>
```

IdP に設定を再読み込みさせます。reload-service.sh は既定のアクセス制御によりログイン画面へリダイレクトされて実行されないことがあるため、学習環境では Tomcat 再起動が確実です。

```
sudo systemctl restart tomcat
sleep 20
# SP メタデータ (entityID: https://sp.plm-lab.local/shibboleth) 読み込み・エラー無しを確認
sudo grep -i 'LocalSP|sp.plm-lab.local' /opt/shibboleth-idp/logs/idp-process.log | tail -n 10
# 期待: "FilesystemMetadataResolver LocalSP: New metadata successfully loaded ..." が出る
```

13.5 時刻同期の最終確認

SAML はクロックスキューに敏感です。WSL2 と Windows の時刻が一致しているか確認します。

```
# WSL2
date

# ゲスト Windows
Get-Date
```

両者が数秒以内で一致していること（フェーズ1 §5.6 の timesyncd と Hyper-V 統合サービスで揃っているはず）。大きくずれている場合は WSL2 で `sudo hwclock -s`。

13.6 初回 SSO の確認

ゲスト Windows のブラウザ（新しいプライベートウィンドウ推奨。既存セッションの影響を避ける）で：

1. `https://sp.plm-lab.local/whoami.asp` を開く
2. **IdP のログイン画面** (`https://idp.plm-lab.local:8443/idp/...`) に遷移する
3. ユーザー名 90001、パスワード（フェーズ3で設定したもの）でログイン
4. **SP に戻り、whoami.asp が表示される**（保護ページに到達）

#	確認内容	期待結果
1	保護の発火+遷移	未認証アクセスで IdP ログイン画面（8443）へ遷移
2	認証	uid=90001 でログインできる（LDAP 認証成立）
3	復路	SP に戻り、whoami.asp が開ける（セッション確立）
4	ステータス	<code>https://localhost/Shibboleth.sso/Session</code> に有効なセッションが見える

この時点で REMOTE_USER は空でも合格。個人番号を REMOTE_USER に載せるのはフェーズ 10。

13.7 つまずいたときの切り分け

- **IdP ログイン画面に飛ばず 443 に行ってしまう** → 13.1 のエンドポイント :8443 補正が未反映。SP 側 `idp-metadata.xml` の Location を確認。
- **IdP 側で「SAML2 SSO profile is not configured for relying party ...sp...」** → IdP に SP メタデータが読めていない（13.4）。`idp-process.log` を確認し、`metadata-providers.xml` のパス・SP メタデータの entityID を確認。
- **署名/復号エラー** → メタデータ内の (b) 証明書と実鍵の不一致。SP/IdP のメタデータが最新か（keygen やインストール後に再取得したか）を確認。
- **クロックスキュー** → 13.5 の時刻確認。
- ログ：SP は `C:\opt\shibboleth-sp\var\log\shibboleth\shibd.log`、IdP は `/opt/shibboleth-idp/logs/idp-process.log`。

13.8 フェーズ 10 への引き継ぎ

初回 SSO が成立したら、残る仕上げは「**個人番号を REMOTE_USER に載せる**」こと。フェーズ 10 で、IdP 側の NameID/属性の解放（uid を出す）と、SP 側の `attribute-map.xml` / REMOTE_USER の対応を設定し、`whoami.asp` に個人番号が表示される状態にする。

14. フェーズ 10：属性連携（個人番号を NameID で渡し REMOTE_USER に載せる）

目的：認証されたユーザーの個人番号（uid）を、IdP から SAML の NameID として SP へ渡し、SP 側で REMOTE_USER にマッピングする。最終的に `https://sp.plm-lab.local/whoami.asp` の REMOTE_USER に **90001** が表示される状態にする（当初方針の (a) NameID 方式）。

方式（再掲）：個人番号は恒久的な識別子なので、一時的な transient NameID ではなく、**uid の値をそのまま NameID として出す**（unspecified 形式）。IdP 側で「uid を解決 → NameID として生成 → 対象 SP へ解放」、SP 側で「その NameID を REMOTE_USER にマップ」する。設定は本フェーズが最も細かいので、**1 か所ずつ変更しては確認**するのが安全。

14.1 IdP：uid 属性の確認（attribute-resolver.xml は変更不要）

IdP 5 の既定の attribute-resolver.xml には、次の定義が**最初から入っています**。

```
<AttributeDefinition id="uid" xsi:type="PrincipalName" />
```

PrincipalName 型は「**ログインした本人の名前**」を値にします。本構成ではユーザーは個人番号（例 90001）でログインするため、この uid の値は**そのまま個人番号**になります。したがって、**attribute-resolver.xml は変更不要**です。

⚠ 注意：同じ id="uid" の AttributeDefinition を重複して追加すると起動エラーになります。LDAP コネクタを別途足す必要はありません（既存の uid=PrincipalName をそのまま NameID のソースに使います）。LDAP から他の属性（氏名・メール等）も取りたい場合のみ、LDAP DataConnector を追加しますが、本構成では個人番号のみなので不要です。

14.2 IdP：uid を対象 SP へ解放（attribute-filter.xml）

/opt/shibboleth-idp/conf/attribute-filter.xml に、SP（sp.plm-lab.local）へ uid を解放するポリシーを追加します。

```
<AttributeFilterPolicy id="releaseUidToPlmSP">
  <PolicyRequirementRule xsi:type="Requester"
    value="https://sp.plm-lab.local/shibboleth"/>
  <AttributeRule attributeID="uid">
    <PermitValueRule xsi:type="ANY"/>
  </AttributeRule>
</AttributeFilterPolicy>
```

14.3 IdP：uid を NameID として生成（saml-nameid.xml）

/opt/shibboleth-idp/conf/saml-nameid.xml の <util:list id="shibboleth.SAML2NameIDGenerators"> に、属性ソースの NameID ジェネレータを追加します（既定の transient 生成の下に足す）。

```
<bean parent="shibboleth.SAML2AttributeSourcedGenerator"
  p:omitQualifiers="true"
  p:format="urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified"
  p:attributeSourceIds="#{ {'uid'} }" />
```

14.4 IdP：対象 SP に unspecified 形式を優先させる（relying-party.xml）

SP が特定の NameID 形式を要求しない場合でも unspecified（uid 由来）を使わせるため、/opt/shibboleth-idp/conf/relying-party.xml の <util:list id="shibboleth.RelyingPartyOverrides"> に、対象 SP 向けのオーバーライドを追加します。

```
<bean parent="RelyingPartyByName"
  c:relyingPartyIds="https://sp.plm-lab.local/shibboleth">
  <property name="profileConfigurations">
    <list>
      <bean parent="SAML2.SSO"
        p:nameIDFormatPrecedence="urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified"/>
    </list>
  </property>
</bean>
```



```
</property>
</bean>
```

設定を反映（Tomcat 再起動が確実）。

```
sudo systemctl restart tomcat
sleep 20
sudo tail -n 40 /opt/shibboleth-idp/logs/idp-process.log # ERROR が無いこと
```

14.5 SP : NameID を REMOTE_USER にマップ

(1) **attribute-map.xml** (C:\opt\shibboleth-sp\etc\shibboleth\attribute-map.xml) に、unspecified 形式の NameID を属性 uid として取り込むデコーダを追加します。

```
<Attribute name="urn:oasis:names:tc:SAML:1.1:nameid-format:unspecified" id="uid">
  <AttributeDecoder xsi:type="NameIDAttributeDecoder" formatter="$Name"/>
</Attribute>
```

(2) **shibboleth2.xml** の <ApplicationDefaults> の REMOTE_USER に、先頭で uid を使うよう変更します。

```
<ApplicationDefaults entityID="https://sp.plm-lab.local/shibboleth"
  REMOTE_USER="uid eppn persistent-id targeted-id">
```

反映（SP 再起動）。

```
C:\opt\shibboleth-sp\sbin\shibd.exe -check # overall configuration is loadable
Restart-Service shibd_Default
iisreset
```

14.6 動作確認（最終ゴール）

ゲスト Windows のブラウザの新しいプライベートウィンドウで：

1. <https://sp.plm-lab.local/whoami.asp> を開く
2. IdP ログイン画面（8443）→ 90001 とパスワードでログイン
3. [whoami.asp](#) に戻り、REMOTE_USER = [90001] と表示される

#	確認内容	期待結果
1	SSO 往復	ログイン後 whoami.asp に戻れる
2	個人番号の受け渡し	REMOTE_USER = [90001]（個人番号が入る）
3	セッション内容	https://localhost/Shibboleth.sso/Session に NameID/属性が見える

これが表示できれば、本手順書の最終目標（個人番号による SSO 連携）は達成です。

14.7 つまずいたときの切り分け

- **REMOTE_USER が空のまま** → ① IdP が NameID を出しているか（[Shibboleth.sso/Session](#) で NameID を確認）、② SP の attribute-map.xml の形式（unspecified）と id（uid）が一致しているか、③ REMOTE_USER の先頭に uid があるか。
- **属性は来るが NameID が transient のまま** → 14.4 の nameIDFormatPrecedence が対象 SP に効いているか。relying-party.xml の entityID を確認。
- **IdP でエラー（uid が解決できない）** → 14.1 の LDAP コネクタ（principalCredential）と、ログイン名が uid と一致しているか。idp-process.log を確認。

- ・ **確認ツール**：ブラウザ拡張「SAML-tracer」で、IdP-SP の SAML Response 内の <NameID> に 90001 が入っているかを直接確認できる。
- ・ **ログ**：SP=C:\opt\shibboleth-sp\var\log\shibboleth\shibd.log、IdP=/opt/shibboleth-idp/logs/idp-process.log。

14.8 実運用（PLM）への接続に向けた補足

本フェーズで REMOTE_USER に個人番号が入るようになった。実際の PLM では、この REMOTE_USER（IIS のサーバ変数）を PLM 側が読み取り、自 DB で認可判定する。従来 Cookie で受けていた個人番号を、この REMOTE_USER 経由に置き換える形になる（PLM 側の受け口の最終確認は継続）。SP はサイト全体を保護しているため、PLM の各ページはすべて認証必須となる。

15. フェーズ 11：結合テスト（SSO の通し確認・ログ・再現性）

目的：構築した SSO を通しで検証し、①ログイン～個人番号の受け渡し、②別ユーザーでの再現性、③ログアウト、④再起動後も動く堅牢性、⑤ログの読み方を確認して、学習環境の構築を完結する。

15.1 ログインの通し確認（クリーンな状態から）

ゲスト Windows のブラウザの**新しいプライベートウィンドウ**で：

1. `https://sp.plm-lab.local/whoami.asp` を開く → IdP ログイン画面（8443）へ遷移
2. 90001 とパスワードでログイン
3. `whoami.asp` に戻り **REMOTE_USER = [90001]** が表示される

15.2 別ユーザーでの再現性

値が固定でなく、認証したユーザーの個人番号が反映されることを確認します。別のプライベートウィンドウで、**90002**（フェーズ 3 で作成した 2 人目）でログインし、**REMOTE_USER = [90002]** になることを確認します。

15.3 セッションの確認

- ・ `https://localhost/Shibboleth.sso/Session ...` 現在のセッションの NameID・属性・IdP entityID などが確認できる。
- ・ `https://localhost/Shibboleth.sso/Status ...` SP の稼働状態（<OK/>）。

15.4 ログアウト

- ・ `https://sp.plm-lab.local/Shibboleth.sso/Logout` にアクセスすると SP セッションが破棄される。
- ・ その後 `whoami.asp` に再アクセスすると、再びログインが要求される（未認証に戻っている）ことを確認。

本構成は SP ローカルのログアウト（Logout>SAML2 Local）。IdP 側のセッションは残るため、ログアウト直後の再ログインでパスワード入力を求められないことがある（IdP セッション有効中のため）。完全に切るにはブラウザを閉じる（プライベートウィンドウを閉じる）のが確実。

15.5 ログの読み方

IdP（WSL2）：/opt/shibboleth-idp/logs/ - idp-process.log ... 一般的な処理ログ。エラー調査の起点。
- idp-audit.log ... **監査ログ**。1行1認証で、いつ・どの利用者が・どの SP へ・何を出したかが分かる

(例：認証時刻、https://sp.plm-lab.local/shibboleth、NameID など)。「誰がログインしたか」を追うのに最適。- idp-warn.log ... 警告のみ抽出。

SP (Windows) : C:\opt\shibboleth-sp\var\log\shibboleth\ - shibd.log ... SP デーモンのログ。アサーション受領・セッション確立・エラーはここ。- transaction.log ... セッション単位の記録。

成功時の典型：IdP の idp-audit.log に 90001 の認証行が出て、SP の shibd.log に「new session created」相当の行が出る。うまくいかないときは、まず IdP 監査ログで「認証・解放まで到達しているか」を見て、IdP 側か SP 側かを切り分ける。

15.6 再起動後の堅牢性 (重要)

学習環境はスリープ無効 (\$1) だが、**再起動しても自動復旧することを確認しておく**と安心です。各サービスが自動起動設定になっているか：

- WSL2 : sudo systemctl is-enabled slapd tomcat apache2 (いずれも enabled)
- Windows : Shibboleth 3 Daemon (自動)、IIS (自動)

ゲスト Windows を再起動 → WSL2 が起動 → 各サービスが上がる → https://sp.plm-lab.local/whoami.asp で再び SSO が通ることを確認します。

mirrored ネットワークや WSL2 のサービスは、wsl --shutdown や再起動のたびに立ち上がるはず。もし WSL2 のサービスが上がっていなければ、sudo systemctl start slapd tomcat apache2 で起動し、is-enabled を確認。時刻ずれが疑われる場合は \$5.6・\$13.5 の要領で確認。

15.7 よくある問題の早見表 (総まとめ)

症状	主な原因	対処 (参照)
ブラウザで IdP に「接続拒否」	localhost 転送が効かない / Apache 停止	mirrored 化 (\$6.3)、Apache 起動、Listen 80/443 無効化 (\$10.4)
Apache が Address already in use	IIS と 80/443 衝突 (mirrored)	Listen 80/443 をコメントアウト (\$10.4)
IdP ログイン後 443 に飛ぶ	IdP メタデータのポート未補正	エンドポイントを :8443 に (\$13.1)
shibd 起動失敗 (content model エラー)	<MetadataProvider> を Sessions 内に配置	</Sessions> の後・<Errors> の下へ (\$13.2)
REMOTE_USER が空	NameID 未生成 / SP マッピング不足	\$14.3~14.5、Session/SAML-tracer で切り分け
clock skew エラー 証明書警告	時刻ずれ rootCA 未信頼	sudo hwclock -s、Hyper-V 時刻同期 (\$5.6) 信頼ルートに登録 (\$6.5)

15.8 構築完了・本番 (PLM) への展開メモ

これで、**個人番号による Shibboleth SSO 連携**の学習環境が完成しました。本番の PLM へ展開する際の要点：

- 保護対象の whoami.asp を **実際の PLM アプリ**に置き換える。PLM は Web サーバ層で確定した REMOTE_USER (個人番号) を読み、自 DB で認可判定する (従来の Cookie 受け渡しの置き換え)。サイト全体保護のため、PLM の各ページは認証必須になる。

- ・ **顧客の IdP** と繋ぐ場合、今回自分で立てた IdP の代わりに顧客 IdP のメタデータを登録し、顧客 IdP が出す識別子 (NameID/属性) と PLM の突合キー (個人番号) が一致することを確認する (フェーズ 9~10 の考え方は同じ)。
- ・ 組織展開の留意点 (本書中で既出) : オフライン導入 (付録 E) 、LDAPS/StartTLS 化 (§9.4) 、秘匿情報の集約 (secrets.properties) 、SP 配置は C:\opt 推奨 (§12.1) 、ビルトイン Administrator の承認モード差異 (§11.5) 、ポートフォワーディング禁止ポリシーの確認 (§6.1.1) 。
- ・ ネットワークは、別マシン (クライアント PC) からのアクセス ((C) 相当) を本番では使う。これは「発展編」として、ホスト名解決を実 IP に向け、rootCA をクライアントに配布し、ファイアウォールで 443/8443 を許可する形になる。

以上でフェーズ 1~11 の全工程が完了です。本書は、同じ手順を組織の検証環境で再現するための土台として利用できます。

付録 A : 評価版の rearm 運用

- ・ 残り日数と rearm 残回数の確認 (ゲスト Windows、管理者) :

`slmgr /dlv`
- ・ 期限が近づいたら延長 (実行後に再起動) :

`slmgr /rearm`
`shutdown /r /t 0`
- ・ `slmgr /rearm` は残り日数を **90 日にリセット** します (加算ではない) 。回数には上限があるため、**期限間際** に実行するほど総稼働日数を最大化できます。
- ・ 本環境は rearm 残回数 2 (執筆時点) 。現在の残日数 + 2 回 × 90 日で、合計おおむね 200 日前後の猶予が見込めます。削除・再インストールは不要です。

付録 B : 作業成果の保全 (VM 消失対策)

- ・ **Hyper-V チェックポイント (運用種別)** : 要所 (各フェーズ完了時) でゲスト VM のチェックポイントを取得。入れ子の親 VM では稼働中のスタンダード (メモリ状態) チェックポイントは不可のため、種類は運用 (Production) に固定し、可能なら VM を停止して取得する (§5.2) 。稼働中に取り場合は直前に `wsl --shutdown` で WSL2 を静止させる。チェックポイントは短期のやり直し用であり、差分ディスク (.avhdx) が増えて性能低下の原因になるため、フェーズが安定したら適用・統合して整理する。
- ・ **WSL2 ディストリのエクスポート** (ゲスト Windows) :

```
# バックアップ
wsl --export Ubuntu-24.04 D:\backup\ubuntu-idp.tar
# 復元 (別環境や再構築時)
wsl --import Ubuntu-24.04 C:\WSL\Ubuntu D:\backup\ubuntu-idp.tar
```

これにより、万一ゲスト Windows を作り直しても、IdP スタックを含む Linux 環境をそのまま復元できます。VM ごと巻き戻すチェックポイントと、Ubuntu 単体を書き出すエクスポートの二重化になります。

付録c：スリープ有効環境向け・復帰時の時刻再同期（参考）

本書はスリープ無効化を前提（\$1）とするため主手順では不要ですが、**スリープ／休止を使う環境**では、復帰後に WSL2 の時計が取り残されて SAML の clock skew エラーを招くことがあります。その場合は、ゲスト Windows の「タスク スケジューラ」で復帰イベントを契機に再同期させます。

1. Win + R → taskschd.msc
2. 「タスクの作成」
 - 全般：名前 WSL2-ClockResync、「最上位の特権で実行する」にチェック
 - トリガー → 新規 → 「タスクの開始：イベント時」
 - ・ ログ：システム / ソース：Kernel-Power / イベント ID：107（スリープ復帰）
 - 操作 → 新規 → 「プログラムの開始」
 - ・ プログラム：wsl.exe
 - ・ 引数：-u root hwclock -s
3. 保存

手動の応急処置（いつでも）：Ubuntu 内で `sudo hwclock -s`。

付録D：トラブルシューティング（随時追記）

事象	主な原因	対処
WSL2 が起動しない	入れ子仮想化が未公開	5.2 の ExposeVirtualizationExtensions を確認（VM 停止中に設定）
VM のメモリ変更が反映されない	入れ子仮想化で稼働中は変更不可	VM を停止してから変更（\$5.2）
SAML で clock skew エラー	WSL2 の時計ズレ	<code>sudo hwclock -s</code> （スリープ運用時は付録Cのタスクを追加）
timedatectl が使えない	systemd 未有効	5.5 を実施し <code>wsl --shutdown</code> 後に再確認
Unknown key 'wsl2.autoMemoryReclaim' 警告	.wslconfig のセクション誤り	autoMemoryReclaim を [experimental] へ移動（\$5.3）

（以降、各フェーズで判明した事象を追記）

付録E：オフライン環境での apt 導入

組織のネットワークでは `apt update`／`apt upgrade` の外部アクセスが制限される場合があります。オフラインでの導入方法を、推奨順に示します。

1. **エクスポート/インポートで持ち込む（最有力）**：ネット接続可能な側で apt 更新・パッケージ導入・IdP スタック構築まで済ませ、Ubuntu を丸ごと持ち込む。組織側では apt を叩かず完成環境を再現できる（本書の「自宅で構築→組織へ反映」と最も噛み合う）。

ネットのある側で書き出し

```
wsl --export Ubuntu-24.04 D:\ubuntu-idp.tar
```

オフライン環境で取り込み

```
wsl --import Ubuntu-24.04 C:\WSL\Ubuntu D:\ubuntu-idp.tar
```

2. **社内ミラー／プロキシを使う**：社内 Ubuntu ミラーや apt プロキシ、HTTP プロキシがある場合はそれを利用する。

HTTP プロキシ経由で apt を使う例

```
echo 'Acquire::http::Proxy "http://<プロキシ>:<ポート>/"'; | sudo tee /etc/apt/apt.conf.d/00proxy
```

社内ミラーがある場合は /etc/apt/sources.list.d/ubuntu.sources の URL を差し替え

3. **.deb を持ち込む（真のオフライン）**：同一版（Ubuntu 24.04 / amd64）の接続機で依存込みにダウンロードし、持ち込んで導入する。個別 dpkg -i は依存で詰まりやすいため、ローカルリポジトリ方式（dpkg-scanpackages で Packages を生成し deb [trusted=yes] file:///path ./ を追加）または apt-offline を推奨。

オンライン側（ダウンロードのみ）

```
sudo apt-get -y install --download-only <パッケージ名>
```

→ /var/cache/apt/archives/*.deb を収集して持ち込み、オフライン側で

```
sudo dpkg -i *.deb ; sudo apt-get -f install # 依存解決
```

注意：アーキテクチャ（amd64）とリリース（noble/24.04）を必ず一致させること。学習・検証用途では1の export/import が最も確実。